Your Images, in Your Hands

Sharing whole imaging studies with a SMART Health Link

One link can carry a complete imaging study: every DICOM file, pixels unchanged, end-to-end encrypted on a host that never holds the key. Any app that can fetch and decrypt can open it.

Josh Mandel · shlink-dicom · October 2026



Open the sample
`shlink-dicom.exe.xyz/sample`

The question

Patients have a legal right to copies of their imaging. In practice a study usually arrives on a CD, or can be looked at in a provider's portal.

SMART Health Links share health records by link or QR code: a URL to an encrypted file plus the key to open it.

SMART Imaging Access lets patients use any app they choose to fetch imaging from their providers through standard APIs (FHIR and DICOMweb).

Can one link carry a whole study, every DICOM file, in a form any app can fetch?

Trust and lifecycle

Sharer's device	Host	Recipient's device
Reads the files, can compress them losslessly, encrypts each study under its own key. Sees the study.	Stores bundle.jwe and one encrypted file per study. Sees random file names, sizes and byte ranges; no names, UIDs or keys.	Opens the link, checks every record, decrypts, decodes. Sees the study.

Plaintext exists only on the two devices. The key travels in the link, after the `#`.

- **Owner link.** No accounts. The owner page shows how often each link was opened, and revokes it: then the server answers 404.
- **Re-share a subset without re-uploading.** Each study has its own key, so a link to one study is just a new bundle.jwe holding that study's key.
- **Storage.** Local disk, or a private S3 bucket. Reads go through the server by default; in redirect mode the presigned URLs last 60 seconds at most.

Standards work

- **Pull request #5** to argonautproject/smart-imaging (open): transfer-syntax negotiation for WADO-RS.
- **Issue #2** (open): the token response example omits token_type and scope.
- **Issue #3** (open): what apps send for transfer syntax, and what happens when it is omitted.
- Building a strict server and client against the guide found **14 places** where it was unclear.
- The profile and its extensions use URLs under the HL7 SMART Health Cards and Links guide, proposed for publication there.

One slice, from link to pixels

Traced by script on the live sample, 16 Oct 2026, 00:58 CDT: brain MRI, series 16, instance 11. The share: 6 encrypted files, one per study, 313.4 MB.

1 SMART Health Link
`…/view#shlink:/eyJ1cmwiOiJodHRwczovL3No…`
After the #, which browsers never send. Holds the JWE's url, a 32-byte key, flag U.

2 JWE
bundle.jwe · 59 KB
alg dir, enc A256GCM, zip DEF. One GET; opened with the link's key.

3 FHIR Bundle
1 Patient · 6 ImagingStudy · 6 DiagnosticReport
Plain FHIR R4: 36 series and 739 DICOM instances in all.

4 ImagingStudy
`linked-file: the study's file id and 32-byte key`
`instance-length: bytes in each instance`
8,768,688 (9 series before series 16) + 3,531,858 (instances 1–10) = 12,300,546, where instance 11 starts.

5 Study file
28,584,796 bytes · 437 records of 64 KiB
series 16: records 133–240 · instance 11: 187–193

6 Records 187–193

187	188	189	190	191	192	193

Each: 65,520 encrypted bytes + a 16-byte tag (dark band).

7 Range request
`GET …/mwh_OLbs1m6cO9BOcfAnxA.bin`
`Range: bytes=12255232-12713983`
→ 206 Partial Content: 458,752 bytes, 7 records.

8 GCM tag check
7 of 7 tags verified
Nonce = record number. A changed, moved or missing record fails; its bytes are never used.

9 DICOM file
Instance 11: 353,408 bytes
`1.2.840.10008.1.2.4.70: JPEG Lossless, SV1`
From byte 48,306 of record 187: preamble, DICM, header, Pixel Data.

10 Pixels



T2 FSE AX, instance 11 of 20
512 × 512, decoded in WebAssembly, windowed on the GPU.

Clients that can follow the same link

The link and files are plain HTTP, FHIR and DICOM. Any program that can fetch, check and decrypt can use them.

built: /view
Browser viewer
Fetches only the records for the images on screen, one Range request each.

built: receiver.js
Download script
About 60 lines of fetch and WebCrypto: resolve the link, decrypt the Bundle, read and check records.

possible, not built here
PACS import
Fetch each study file whole, check every tag, write the original DICOM files into the archive.

possible, not built here
Research pipeline
Read the Bundle's index, fetch just the series it needs, and decode them with its own tools.

What it does

Viewer

- CT and MR reformats: oblique planes, slabs
- X-rays with their VOI LUTs
- Whole-slide pathology, tile by tile
- Video, including MPEG-2
- Segmentations
- Encapsulated PDF
- Reports beside the images
- Download: a ZIP with DICOMDIR
- Dark mode
- Phone layout

Sharing

- Files, a folder, a CD, an archive
- From a link someone sent
- From a provider, by SMART Imaging Access
- Lossless JPEG or JPEG 2000
- Resumable uploads
- Local disk or S3 storage
- Owner link: open counts, revoke

Evidence from real files

Decoders, against reference decoders

1,000 / 1,000 random IDC files decoded exactly as the references do (95% interval 99.62–100%)

9,522 / 9,537 IDC files picked for rare scanners, software and encodings passed. The 15 others: 8 lack Planar Configuration (now inferred), 4 Leica JPEG 2000 off by ≤3 (ruled acceptable), 3 float maps (now decoded).

832 real-image cases in the decoder test corpus, all passing

867 / 885 renderings of real files on which dcmtk agrees within one grey level. The 18 others: 7 from our VOI LUT bit-depth rule, 8 with no stored window, 2 off by two levels, 1 where pydicom sides with us.

Lossless encoders

2.46× smaller with lossless JPEG (SV1) on Josh's five studies, 403 → 164 MB. Every image decodes bit-exact in libjpeg, GDCM and dcmtk.

2.96× with lossless JPEG 2000, 403 → 136 MB; bit-exact under three OpenJPEG-based decoders.

Scripted browser checks

Against the live sites: How it works 254 / 255 (the miss: one 109 ms main-thread task, limit 100 ms) · SMART Imaging sandbox 153 / 153.

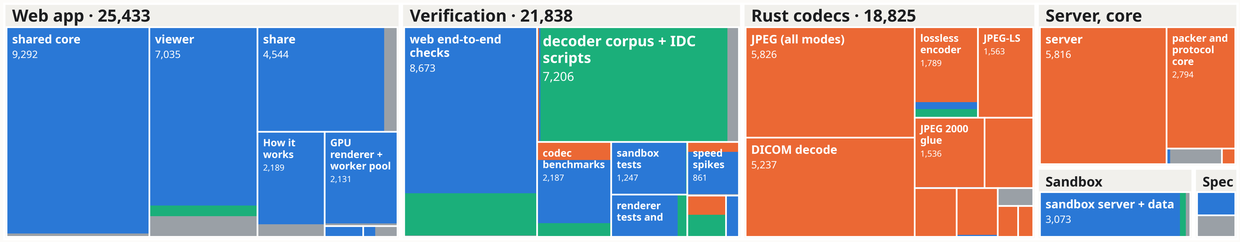
Object types, viewed end to end

120 / 156 cases pass, across 43 object types: all in IDC, plus five it lacks. 22 fail, from seven defects being fixed; 14 are labelled not displayed, with a download.

IDC has no Enhanced or Legacy Converted CT, MR or PET, so those came from public test files and from series derived from IDC.

The code, by job and language

Rust · TypeScript / JavaScript · Python · Other (SQL, C, shell, CSS, HTML, Markdown)



Web app 25,433 · **Verification** 21,838 · **Rust codecs** 18,825 · **Server and protocol core** 8,970 · **SMART Imaging sandbox** 3,073 · **Spec and reference** 879
79,018 non-blank lines of the project's own code on main (04b92d6). The generated DICOM dictionary is not counted.

JPEG 2000 decoding uses hayro-jpeg2000, vendored with our patches, among them HTJ2K decoding built on ndic-htj2k's block decoder. oxideav-jpeg2000 decodes the rare HTJ2K files hayro declines, and cross-checks it. JPEG XL uses the jxl-oxide library. Everything else, including both encoders, is this project's code.

How it was built

1 Claude Code session as coordinator, which owned the shared files, ran the checks and made the commits · **47** Claude subagents, each owning a directory · **1** pi session running Gemini, which built the first server and web page · **1** Codex session for the illustrations

96 commits on main, first to latest



1. **Wed 15:13** First commit: the proposal
2. **Thu 07:26** Pure-Rust decoders, test corpus, renderer
3. **Thu 08:18** Viewer, share and How pages; SMART Imaging sandbox
4. **Thu 08:55** Whole-slide viewer; deploy scripts
5. **Thu 10:21** IDC long-tail sweep
6. **Thu 10:44** Sample data: Josh Example
7. **Thu 12:20** Decoders: float, every JPEG mode
8. **Thu 13:22** Resumable uploads
9. **Thu 15:47** One encrypted file per study
10. **Thu 16:13** Local or S3 storage
11. **Thu 17:10** Display checked against dcmtk and pydicom
12. **Thu 18:37** Phone layout
13. **Thu 19:54** Sandbox: transfer-syntax negotiation (PR #5)
14. **Thu 20:07** Lossless compression when sharing
15. **Thu 20:15** Lossless JPEG encoder and transcoder

Questions that removed design layers

> *"What's the zip buying us?"*
> No ZIP container: offsets in the ImagingStudy and the GCM tags already locate and protect every byte.

> *"I don't love optionality here, how does a client even know what's deflated."*
> No optional compression layer: DICOM already carries its own compression, so every client has one code path.

> *"Final record delimiter.. why is that needed? Doesn't ending at that point also accomplish the same?"*
> No end-of-file marker: the Bundle states every length, so a cut-short file shows up in the arithmetic.

> *"Why use HKDF at that point?"*
> No key derivation: each file has its own random key, carried in the Bundle.

Links · site `shlink-dicom.exe.xyz` · sample `shlink-dicom.exe.xyz/sample` · code and spec `github.com/jmandel/shlink-dicom` (PROPOSAL.md) · SMART Imaging sandbox `smart-imaging-sandbox.exe.xyz`
Data · MRI, CT and X-rays: Josh Mandel's own, de-identified as Josh Example · lung slide: CPTAC-LUAD, CC BY 4.0 · shlink-dicom is a tech demo